Algorithms

Lecture # 29

Syed Hamed Raza

Shortest Path

Shortest Path

To motivate our first algorithm on graphs, consider the following problem.

- We are given an undirected graph $G = (V, E)$ and a source vertex $s \in V$.
- The length of a path in a graph is the number of edges on the path.
- We would like to find the shortest path from s to each other vertex in the tree.

Shortest Path

Shortest Path

The final result will be represented in the following way.

- For each vertex $v \in V$, we will store d[v] which is the distance (length of the shortest path) from s to v. Note that d[s] = 0.
- We will also store a predecessor (or parent) pointer π[v] which is the first vertex along the shortest path if we walk from v backwards to s. We will set π[s] = Nil.

Shortest Path

Shortest Path

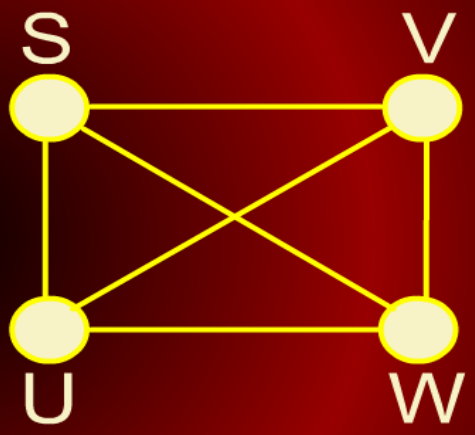
- There is a simple brute-force strategy for computing shortest paths.
- We could simply start enumerating all simple paths starting at s, and keep track of the shortest path arriving at each vertex.
- However, there can be as many as n! simple paths in a graph.
- Clearly this is not feasible.

Shortest Path

Shortest Path

Consider a fully connected graph

- n choices for source node s.
- (n - 1) choices for destination node.
- (n - 2) for first hop (edge) in the path, (n - 3) for second, (n - 4) for third down to (n - (n - 1)) for last leg.
- This leads to n! simple paths



Breadth-first Search

Breadth-first Search

Here is a more efficient algorithm called the breadth-first search (BFS)

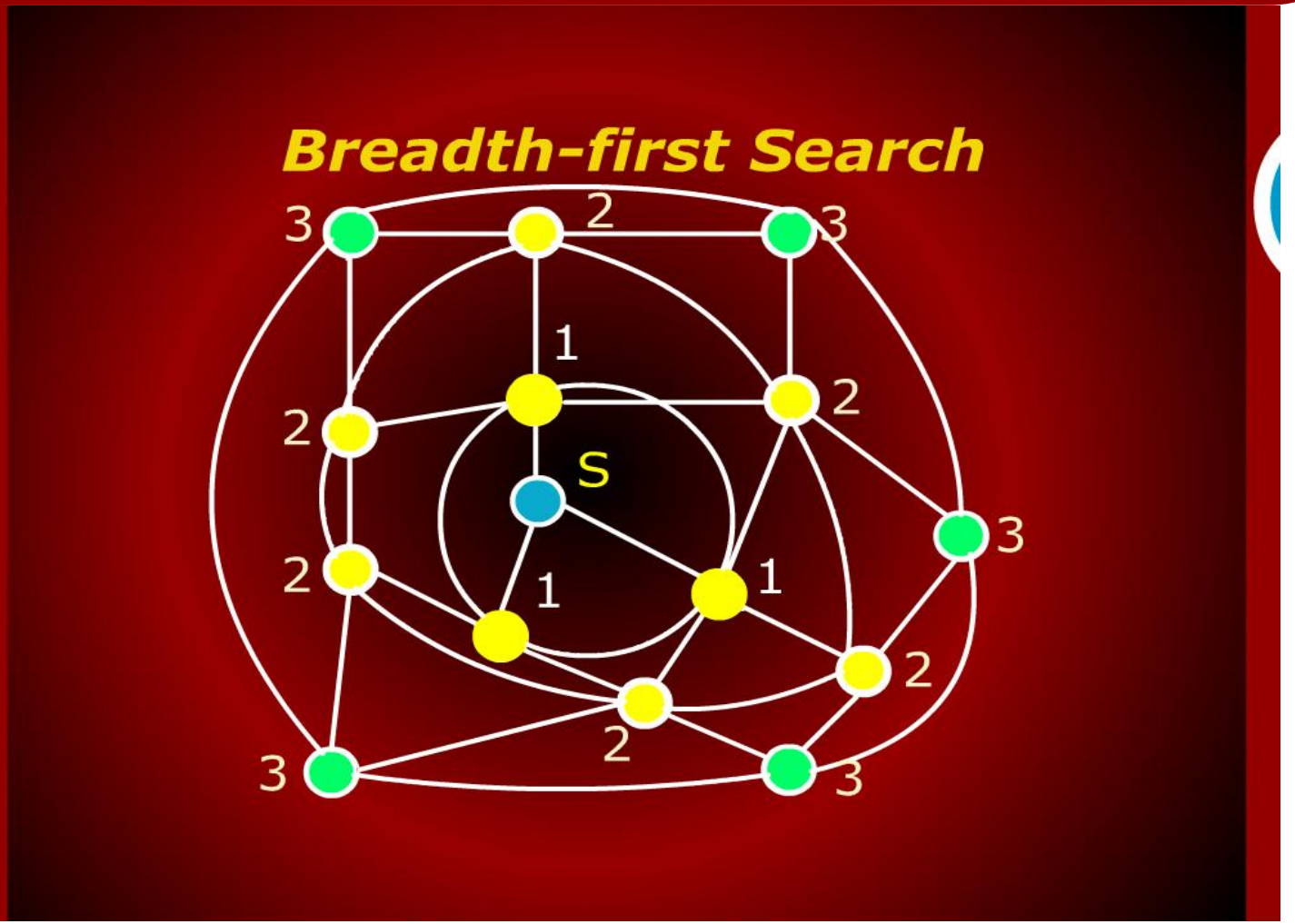
- Start with s and visit its adjacent nodes.
- Label them with distance 1.
- Now consider the neighbors of neighbors of s. These would be at distance 2.
- Now consider the neighbors of neighbors of neighbors of s. These would be at distance 3.

Breadth-first Search

Breadth-first Search

- Repeat this until no more unvisited neighbors left to visit.
- The algorithm can be visualized as a wave front propagating outwards from s visiting the vertices in bands at ever increasing distances from s.

Breadth-first Search



Traversing Connected Graphs

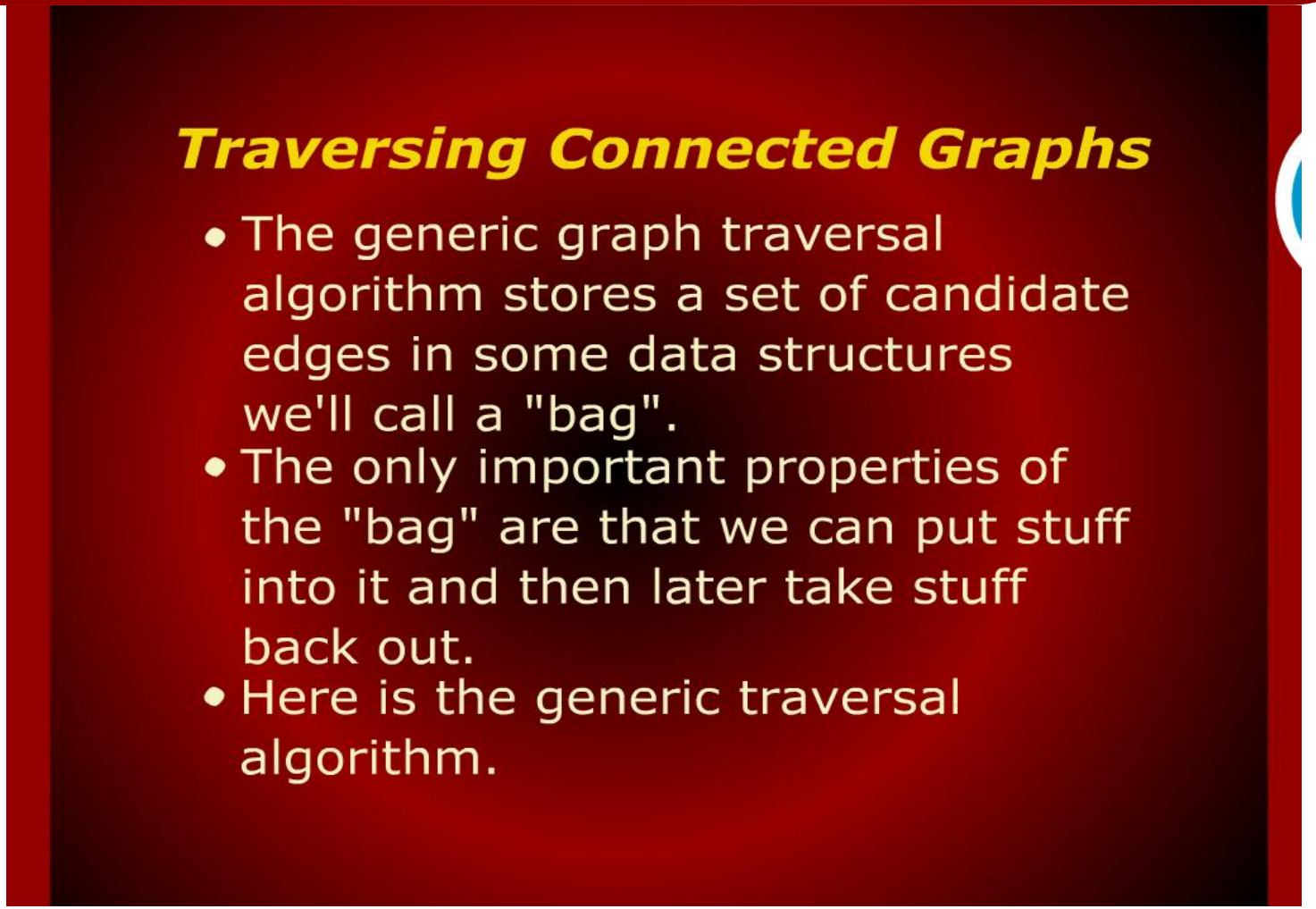
Traversing Connected Graphs

- Breadth-first search is one instance of a general family of graph traversal algorithms.
- Traversing a graph means visiting every node in the graph.
- Another traversal strategy is *depth-first search* (DFS).
- DFS procedure can be written recursively or non-recursively.
- Both versions are passed s initially.

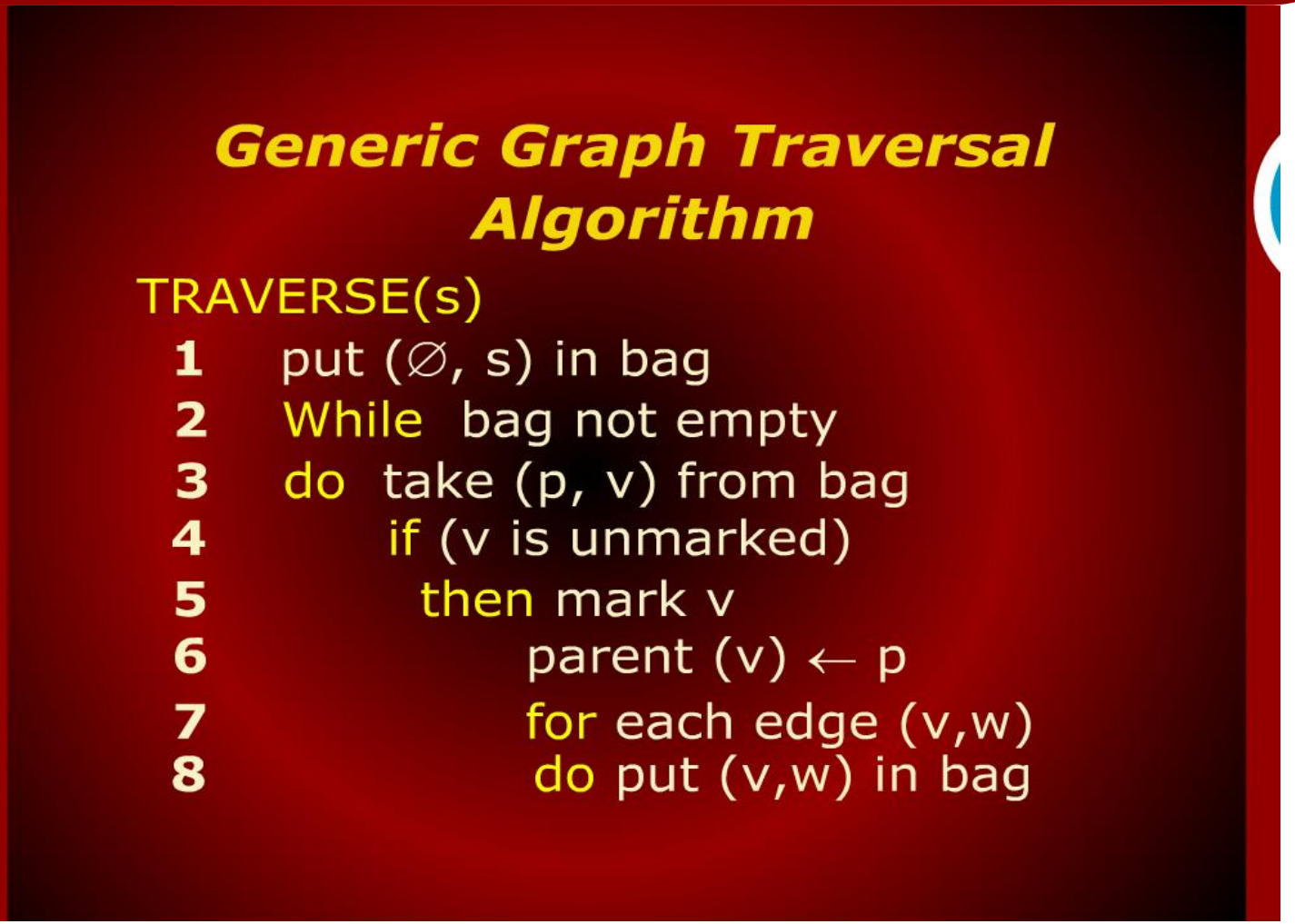
Depth-first Search

Depth-first Search

Traversing Connected Graphs



Generic Graph Traversal Algorithm

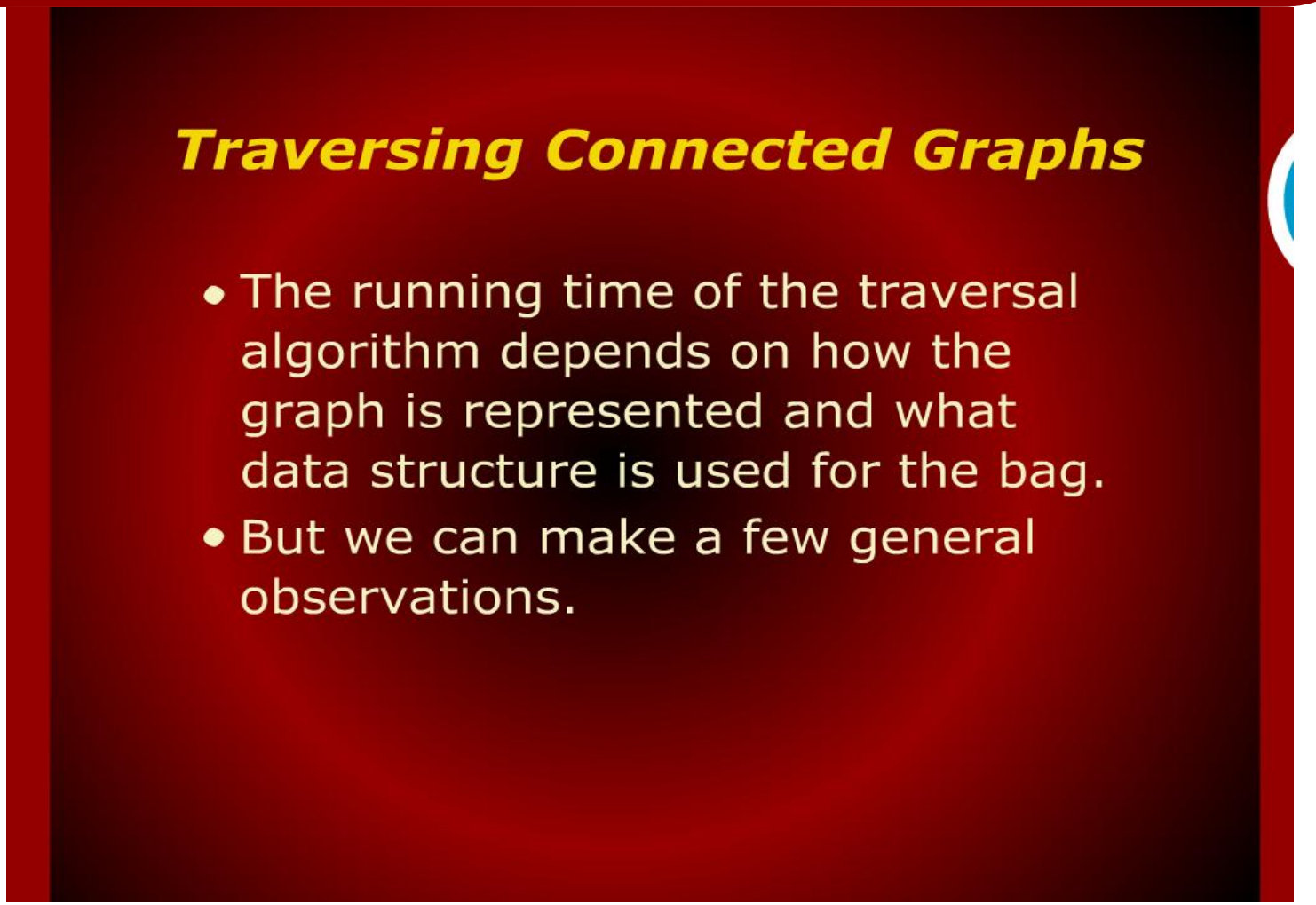


Traversing Connected Graphs

Traversing Connected Graphs

- Notice that we are keeping edges in the bag instead of vertices.
- This is because we want to remember, whenever we visit v for the first time, which previously-visited vertex p put v into the bag.
- The vertex p is call the parent of v.

Traversing Connected Graphs



Generic Graph Traversal Algorithm

Generic Graph Traversal Algorithm

- Since each vertex is visited at most once, the for loop in line 7 is executed at most V times.
- Each edge is put into the bag exactly twice; once as (u, v) and once as (v, u), so line 8 is executed at most 2E times.

```
TRAVERSE(s)
1  put (∅, s) in bag
2  While  bag not empty
3  do  take (p, v) from bag
4   if (v is unmarked)
5       then mark v
6            parent (v) ← p
7            for each edge (v,w)
8            do put (v,w) in bag
```

Generic Graph Traversal Algorithm

Generic Graph Traversal Algorithm

Finally, since we can't take out more things out of the bag than we put in, line 3 is executed at most 2E + 1 times.

```
TRAVERSE(s)
1  put (∅, s) in bag
2  While  bag not empty
3  do  take (p, v) from bag
4   if (v is unmarked)
5       then mark v
6            parent (v) ← p
7            for each edge (v,w)
8            do put (v,w) in bag
```

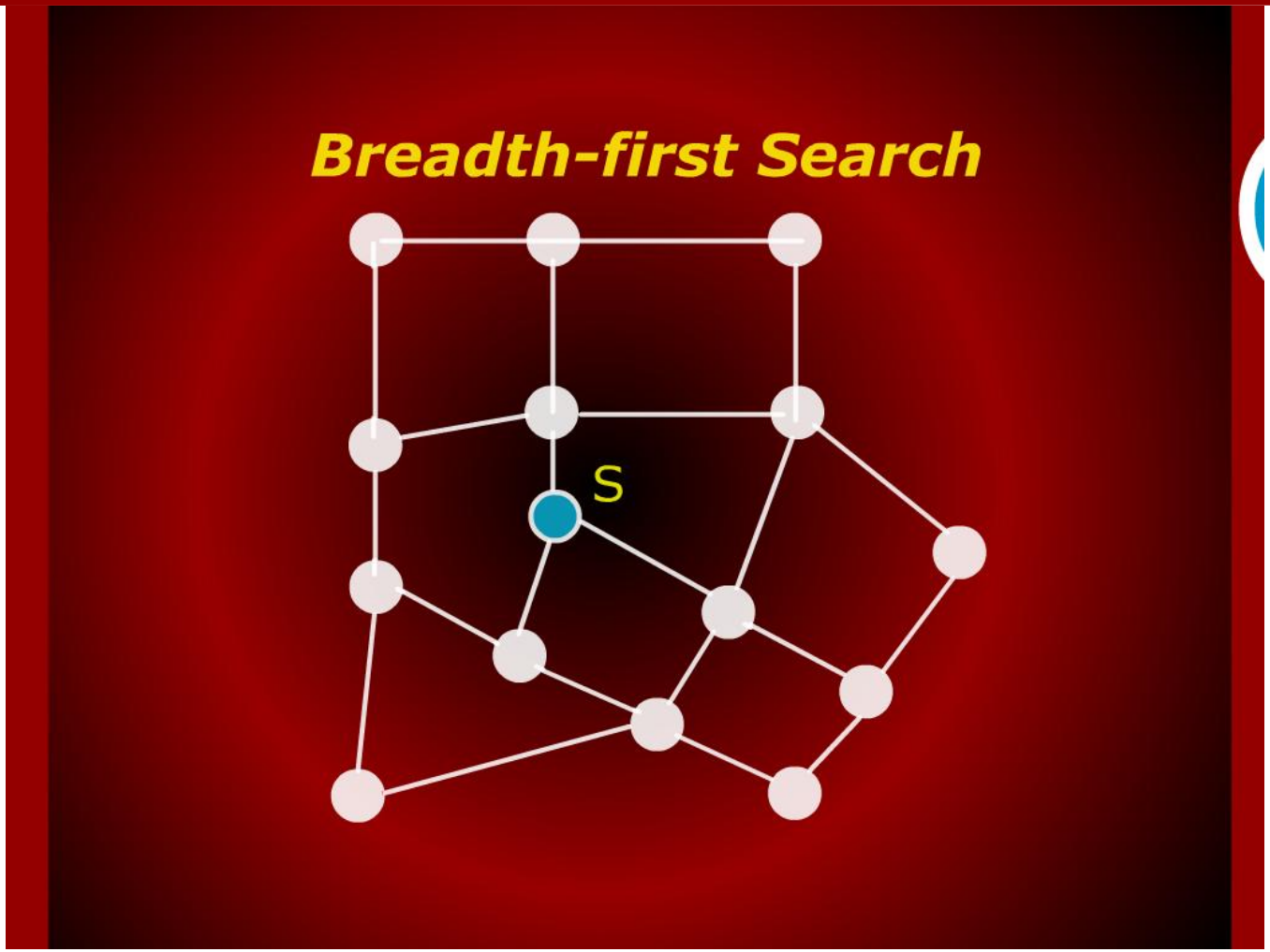
Generic Graph Traversal Algorithm

Generic Graph Traversal Algorithm

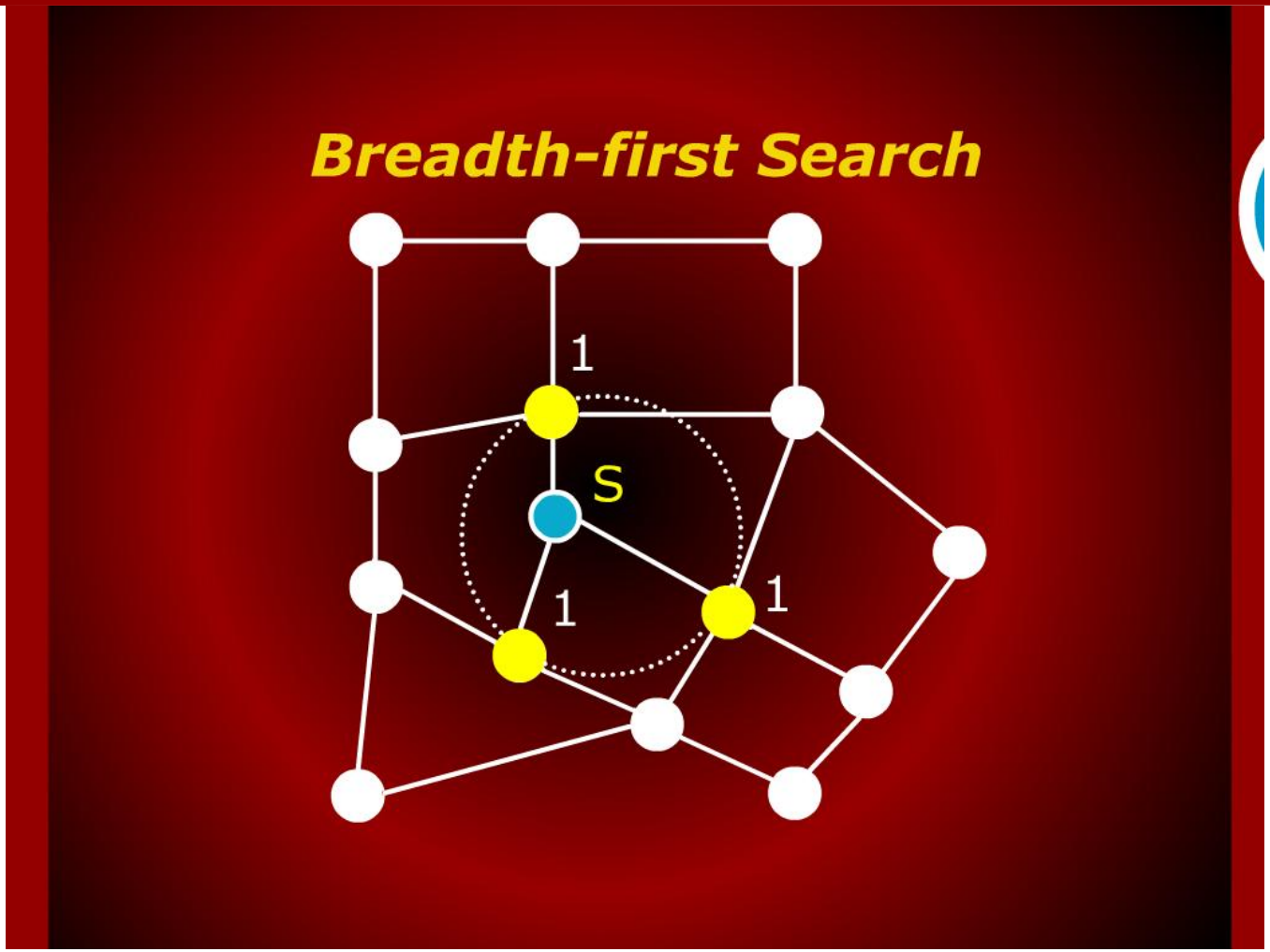
- Assume that the graph is represented by an adjacency list so the overhead of the for loop in line 7 is constant per edge.
- If we implement the bag by using a stack, we have depth-first search (DFS) or traversal.

```
TRAVERSE(s)
1  push (∅, s)
2  Whil  stack not empty
3  do  pop(p, v)
4   if (v is unmarked)
5       then mark v
6            parent (v) ← p
7            for each edge (v,w)
8            do push (v,w)
```

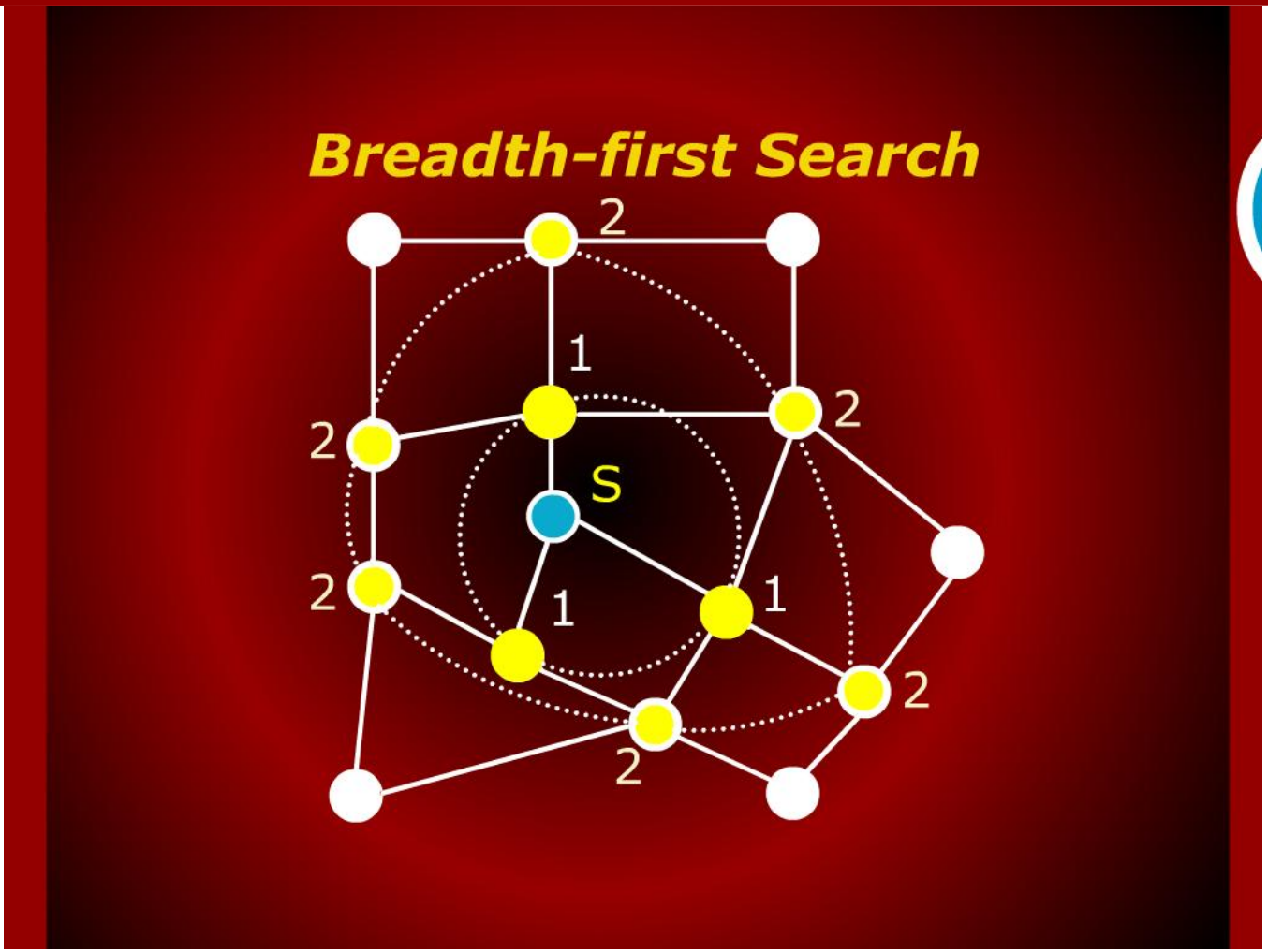
Breadth-first Search



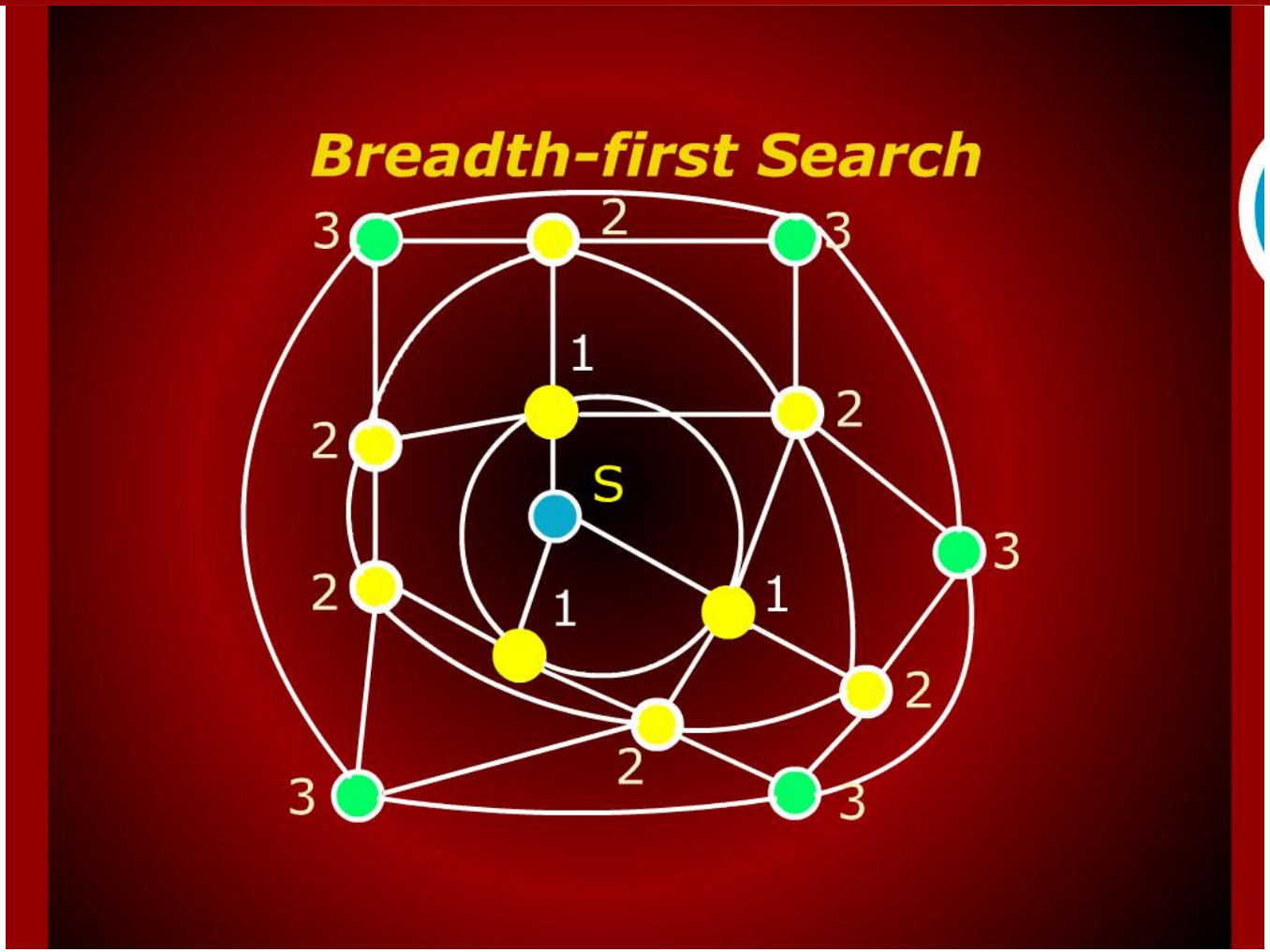
Breadth-first Search



Breadth-first Search



Breadth-first Search



GRAPHS

Depth-first Search

Breadth-first Search